

Qt Base Course

Day 3

Supporting Techniques

Day 3 — Agenda

Supporting Techniques

- Qt container classes and iteration methods
- SQL access — `QSqlDatabase` , `QSqlQuery` , SQL models
- XML processing — StreamReader, SAX, DOM
- JSON processing
- Thread programming variants
- Inter-thread communication and synchronisation

Part 1 — Qt Container Classes

Qt Containers vs. STL

Qt ships its own container library with **implicit sharing (copy-on-write)**:

Qt	STL equivalent	Notes
<code>QList<T></code>	<code>std::vector<T></code>	Qt 6: unified with <code>QVector</code>
<code>QMap<K, V></code>	<code>std::map<K, V></code>	Red-black tree, sorted keys
<code>QHash<K, V></code>	<code>std::unordered_map<K, V></code>	O(1) average lookup
<code>QSet<T></code>	<code>std::unordered_set<T></code>	Hash-based unique elements
<code>QMultiMap<K, V></code>	<code>std::multimap<K, V></code>	Multiple values per key
<code>QPair<A, B></code>	<code>std::pair<A, B></code>	Two heterogeneous values

Qt containers integrate with `QVariant`, `QDebug`, and Qt's serialisation — use them when interacting with Qt APIs.

QList / QVector

```
QList<QString> fruits = {"Apple", "Banana", "Cherry"};

fruits.append("Date");
fruits.prepend("Açaí");
fruits.insert(2, "Blueberry");

QDebug() << fruits.size();           // 6
QDebug() << fruits.at(0);             // "Açaí"
QDebug() << fruits.contains("Banana"); // true
QDebug() << fruits.indexOf("Cherry"); // 3

fruits.removeAll("Banana");
fruits.sort();                       // lexicographic sort
```

QMap and QHash

```
// QMap – sorted by key, O(log n) access
QMap<QString, int> scores;
scores["Alice"] = 95;
scores["Bob"]   = 87;
scores["Carol"] = 92;

qDebug() << scores.value("Bob", 0);    // 87 (default = 0 if missing)
qDebug() << scores.keys();              // ["Alice", "Bob", "Carol"] – sorted

// QHash – faster, O(1) average, unordered
QHash<QString, int> fastLookup;
fastLookup.insert("Alice", 95);
fastLookup.insert("Bob",   87);

if (fastLookup.contains("Alice"))
    qDebug() << fastLookup["Alice"]; // 95
```

Iteration Methods

```
QList<int> numbers = {1, 2, 3, 4, 5};

// 1. Range-based for (preferred)
for (int n : numbers)
    qDebug() << n;

// 2. STL-style iterators (read-write)
for (auto it = numbers.begin(); it != numbers.end(); ++it)
    *it *= 2;

// 3. Java-style iterator (read-only)
QListIterator<int> it(numbers);
while (it.hasNext())
    qDebug() << it.next();

// 4. Mutable Java-style iterator (modify while iterating)
QMutableListIterator<int> mit(numbers);
while (mit.hasNext())
    if (mit.next() % 2 == 0)
        mit.remove();
```

Implicit Sharing (Copy-on-Write)

```
QList<int> a = {1, 2, 3};  
QList<int> b = a;           // 0(1) – shared internal buffer, no copy yet  
  
b.append(4);               // copy triggered here – a unaffected  
  
qDebug() << a.size();     // 3  
qDebug() << b.size();     // 4
```

- Passing containers **by value** to functions is safe and cheap
- Actual copying only happens when one copy is modified
- All Qt value types (`QString` , `QByteArray` , `QImage` , ...) use the same mechanism

Part 2 — SQL Access

Qt SQL Module Overview

```
find_package(Qt6 REQUIRED COMPONENTS Sql)
target_link_libraries(MyApp PRIVATE Qt6::Sql)
```

Supported drivers (built-in or plugin):

Driver	Database
QSQLITE	SQLite 3 — bundled, no server needed
QPSQL	PostgreSQL
QMYSQL	MySQL / MariaDB
QODBC	Any ODBC-compliant database

QSqlDatabase — Opening a Connection

```
// SQLite (file-based, ideal for local apps)
QSqlDatabase db = QSqlDatabase::addDatabase("QSQLITE");
db.setDatabaseName("app.db");

if (!db.open()) {
    qCritical() << "Cannot open database:" << db.lastError().text();
    return;
}

// PostgreSQL (named connection — allows multiple connections)
QSqlDatabase pg = QSqlDatabase::addDatabase("QPSQL", "reporting");
pg.setHostName("db.example.com");
pg.setDatabaseName("analytics");
pg.setUserName("reader");
pg.setPassword(getPasswordFromKeychain()); // never hardcode passwords
pg.open();
```

QSqlQuery — Executing Queries

```
QSqlQuery q;

// DDL
q.exec("CREATE TABLE IF NOT EXISTS users "
      "(id INTEGER PRIMARY KEY, name TEXT, score INTEGER)");

// Parameterised INSERT — prevents SQL injection
q.prepare("INSERT INTO users (name, score) VALUES (:name, :score)");
q.bindValue(":name", "Alice");
q.bindValue(":score", 95);
q.exec();

// SELECT and iterate
q.exec("SELECT id, name, score FROM users ORDER BY score DESC");
while (q.next()) {
    int id = q.value("id").toInt();
    QString name = q.value("name").toString();
    int score = q.value("score").toInt();
    qDebug() << id << name << score;
}
```

SQL Injection — Always Use Parameterised Queries

```
// ✗ NEVER do this – SQL injection vulnerability
QString name = lineEdit->text();
q.exec("SELECT * FROM users WHERE name = '" + name + "'");
// If name = "'; DROP TABLE users;--" → catastrophic

// ✔ Correct – parameterised, safe
q.prepare("SELECT * FROM users WHERE name = :name");
q.bindValue(":name", lineEdit->text());
q.exec();
```

Qt's `:name` syntax protects against injection by treating bound values as data, never as SQL syntax.

QSqlTableModel

```
auto *model = new QSqlTableModel(this, db);
model->setTable("users");
model->setEditStrategy(QSqlTableModel::OnManualSubmit);
model->select();    // execute SELECT * FROM users

// Bind to a view
auto *view = new QTableView(this);
view->setModel(model);
view->hideColumn(0);    // hide the id column

// Filter and sort
model->setFilter("score > 80");
model->setSort(2, Qt::DescendingOrder);
model->select();    // re-execute with new filter/sort

// Commit edits made in the view
model->submitAll();
```

Part 3 — XML Processing

Three Approaches to XML

Approach	API	Memory	Use when
Streaming	<code>QXmlStreamReader</code>	$O(1)$ — stream	Large files, read-only, sequential
SAX	<code>QXmlSimpleReader</code> + <code>QXmlContentHandler</code>	$O(1)$ — events	Legacy API, event-driven
DOM	<code>QDomDocument</code>	$O(n)$ — whole tree in RAM	Random access, small files, modify & write

QXmlStreamReader — Fast Streaming Parser

```
QFile file("config.xml");
file.open(QIODevice::ReadOnly);

QXmlStreamReader xml(&file);
while (!xml.atEnd() && !xml.hasError()) {
    QXmlStreamReader::TokenType token = xml.readNext();

    if (token == QXmlStreamReader::StartElement) {
        if (xml.name() == QLatin1String("item")) {
            QString id = xml.attributes().value("id").toString();
            QString name = xml.readElementText();
            qDebug() << id << name;
        }
    }
}
if (xml.hasError())
    qWarning() << "XML error:" << xml.errorString();
```

DOM Parsing with QDomDocument

```
QDomDocument doc;
QFile file("config.xml");
file.open(QIODevice::ReadOnly);
doc.setContent(&file);    // whole file parsed into memory

QDomElement root = doc.documentElement();    // <config>
QDomNodeList items = root.elementsByTagName("item");

for (int i = 0; i < items.count(); ++i) {
    QDomElement el = items.at(i).toElement();
    qDebug() << el.attribute("id") << el.text();
}

// Modify and write back
QDomElement newItem = doc.createElement("item");
newItem.setAttribute("id", "42");
newItem.appendChild(doc.createTextNode("New entry"));
root.appendChild(newItem);

QFile out("config_updated.xml");
out.open(QIODevice::WriteOnly);
QTextStream stream(&out);
doc.save(stream, 4);    // 4-space indent
```

Part 4 — JSON Processing

QJsonDocument — Parse and Serialise

```
// Parse from bytes (e.g., HTTP response, file)
QByteArray raw = R"({"name":"Alice","score":95,"tags":["qt","cpp"]})";

QJsonParseError err;
QJsonDocument doc = QJsonDocument::fromJson(raw, &err);

if (err.error != QJsonParseError::NoError) {
    qWarning() << "JSON parse error:" << err.errorString();
    return;
}

// Serialise back to compact or indented JSON
QByteArray compact = doc.toJson(QJsonDocument::Compact);
QByteArray indented = doc.toJson(QJsonDocument::Indented);
```

QJsonObject and QJsonArray

```
QJsonObject obj = doc.object();

QString name  = obj["name"].toString();
int          score = obj["score"].toInt();

QJsonArray tags = obj["tags"].toArray();
for (const QJsonValue &tag : tags)
    qDebug() << tag.toString();

// Build JSON programmatically
QJsonObject result;
result["status"]  = "ok";
result["count"]   = 42;
result["items"]   = QJsonArray({ "one", "two", "three" });

QJsonDocument outDoc(result);
QByteArray payload = outDoc.toJson(QJsonDocument::Compact);
// → {"count":42,"items":["one","two","three"],"status":"ok"}
```

Part 5 — Thread Programming

Why Threads in Qt?

The UI runs on the **main thread** — any long-running operation blocks it:

```
// ❌ This freezes the UI for several seconds
void MainWindow::onLoadClicked() {
    QByteArray data = downloadFile("https://example.com/large.zip");
    processData(data);
}
```

Solution: move the work to a background thread, communicate back via signals.

Rule: **only the main thread** may touch UI widgets. All other threads must use signals/queued connections.

Thread Programming Variants

API	Abstraction	Best for
<code>QThread</code> + worker object	Low-level, explicit	Long-running services, custom event loops
<code>QThread</code> subclass	Low-level	Simple, stateless background tasks
<code>QtConcurrent::run</code>	High-level	Fire-and-forget functions
<code>QRunnable</code> / <code>QThreadPool</code>	Medium	Many short tasks, reuse thread pool
<code>QFuture</code> / <code>QFutureWatcher</code>	High-level result	Async result with progress

QThread — Worker Object Pattern (Recommended)

```
class Worker : public QObject {
    Q_OBJECT
public slots:
    void doWork(const QString &url) {
        QByteArray data = downloadFile(url);    // blocks, but on worker thread
        emit resultReady(data);
    }
signals:
    void resultReady(const QByteArray &data);
};

// In MainWindow:
auto *thread = new QThread(this);
auto *worker = new Worker;                // no parent — will be moved
worker->moveToThread(thread);

connect(thread, &QThread::finished, worker, &QObject::deleteLater);
connect(this, &MainWindow::startWork, worker, &Worker::doWork);
connect(worker, &Worker::resultReady, this, &MainWindow::handleResult);

thread->start();
emit startWork("https://example.com/data.zip");
```

QThread — Subclassing Approach

```
class HashThread : public QThread {
    Q_OBJECT
public:
    explicit HashThread(const QString &path, QObject *parent = nullptr)
        : QThread(parent), m_path(path) {}

signals:
    void hashReady(const QByteArray &hash);

protected:
    void run() override {                // called on the new thread
        QCryptographicHash h(QCryptographicHash::Sha256);
        QFile f(m_path);
        f.open(QIODevice::ReadOnly);
        h.addData(&f);
        emit hashReady(h.result());    // QueuedConnection to main thread
    }

private:
    QString m_path;
};
```

QtConcurrent::run — Simplest Approach

```
#include <QtConcurrent/QtConcurrent>

// Run a lambda or function in the global thread pool
QFuture<QByteArray> future = QtConcurrent::run([](const QString &path) {
    QFile f(path);
    f.open(QIODevice::ReadOnly);
    return f.readAll();
}, filePath);

// Watch for completion – back on the main thread
auto *watcher = new QFutureWatcher<QByteArray>(this);
connect(watcher, &QFutureWatcher<QByteArray>::finished, this, [watcher]() {
    QByteArray data = watcher->result();
    // safe to update UI here
    watcher->deleteLater();
});
watcher->setFuture(future);
```

Part 6 — Inter-Thread Communication & Synchronisation

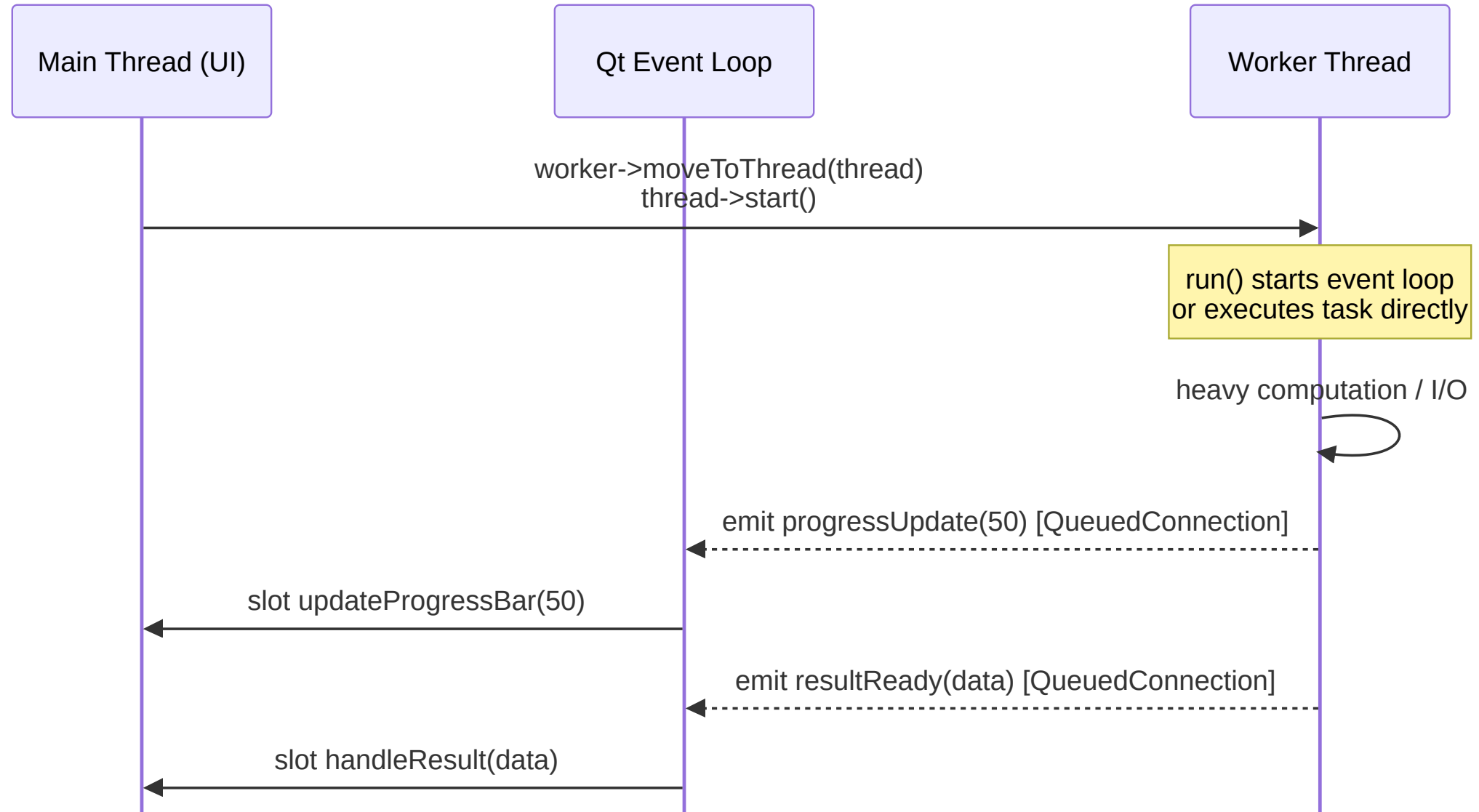
Signals and Slots Across Threads

Qt automatically selects the right connection type:

Connection Type	When	Behaviour
<code>Qt::AutoConnection</code>	Default	Direct if same thread, queued if different
<code>Qt::DirectConnection</code>	Explicitly forced	Slot runs on sender's thread — dangerous across threads
<code>Qt::QueuedConnection</code>	Explicitly forced	Slot posted as event, runs on receiver's thread event loop
<code>Qt::BlockingQueuedConnection</code>	Explicitly forced	Sender blocks until slot returns — can deadlock

```
// Force queued even if same thread
```

Thread Communication Flow



QMutex and QMutexLocker

```
class SharedCounter {
public:
    void increment() {
        QMutexLocker locker(&m_mutex);    // locks on construction
        ++m_count;
    }                                     // unlocks automatically on destruction

    int count() const {
        QMutexLocker locker(&m_mutex);
        return m_count;
    }

private:
    mutable QMutex m_mutex;
    int m_count = 0;
};
```

Always use `QMutexLocker` — it unlocks even if an exception is thrown.

QSemaphore — Counting Semaphore

```
// Producer-consumer: buffer of 10 slots
QSemaphore freeSlots(10);    // 10 slots available
QSemaphore usedSlots(0);     // 0 items to consume

// Producer thread
void produce(const QByteArray &item) {
    freeSlots.acquire(1);     // wait for a free slot
    buffer.enqueue(item);
    usedSlots.release(1);     // signal one item available
}

// Consumer thread
QByteArray consume() {
    usedSlots.acquire(1);     // wait for an item
    QByteArray item = buffer.dequeue();
    freeSlots.release(1);     // signal a slot is free
    return item;
}
```


QWaitCondition

```
QMutex mutex;  
QWaitCondition dataReady;  
QByteArray sharedData;  
bool hasData = false;  
  
// Worker thread – signals when data is ready  
{  
    QMutexLocker locker(&mutex);  
    sharedData = fetchData();  
    hasData = true;  
    dataReady.wakeAll();    // wake all waiting threads  
}  
  
// Consumer thread – waits until data is ready  
{  
    QMutexLocker locker(&mutex);  
    while (!hasData)  
        dataReady.wait(&mutex);    // atomically releases mutex and waits  
    process(sharedData);  
}
```

QReadWriteLock

```
// Multiple readers, exclusive writers
QReadWriteLock lock;
QStringList cache;

// Read – many threads can read concurrently
void readCache() {
    QReadLocker locker(&lock);           // shared read lock
    qDebug() << cache;
}

// Write – exclusive access, blocks all readers
void writeCache(const QStringList &newData) {
    QWriteLocker locker(&lock);          // exclusive write lock
    cache = newData;
}
```

Use `QReadWriteLock` when reads are frequent and writes are rare — higher throughput than a plain `QMutex`.

Atomic Operations

```
#include <QAtomicInt>
#include <QAtomicPointer>

// Lock-free reference counting
QAtomicInt refCount(1);

refCount.ref();      // thread-safe increment
if (!refCount.deref()) {
    // thread-safe decrement – returns false when count reaches 0
    delete this;
}

// Atomic compare-and-swap
int expected = 5;
refCount.testAndSetOrdered(expected, 10); // only sets to 10 if value == 5
```

Atomic operations avoid mutex overhead for simple counters and flags — but do not replace mutexes for protecting compound data structures.

Day 3 Summary

Topic	Key Takeaway
Qt Containers	Implicit sharing makes value semantics cheap; use Qt containers for Qt API integration
SQL	Always use parameterised queries; SQL models connect directly to views
XML	StreamReader for large/streaming; DOM for small files needing random access
JSON	<code>QJsonDocument</code> → <code>QJsonObject</code> / <code>QJsonArray</code> for parsing and building
Threads	Worker object + <code>moveToThread</code> is the recommended pattern
	<code>QMutexLocker</code> for mutual exclusion; <code>QReadWriteLock</code> for read-